



POLICE DEPARTMENT

TERP PROTECT



A database system designed to manage, analyze, and maintain the arrest records of the University of Maryland Police Department efficiently and transparently.

Team Members:

Hrishad Amre

Dhvani Khatri

Rishabh Lnu

Kushal Gogi

Date- 11th October 2025

BACKGROUND

The system integrates all arrest-related data that was previously scattered across multiple sources into one centralized database.

Users:

- UMPD Officers – Record and update arrest information
- Analysts / Supervisors – Track trends, analyze cases, and evaluate officer performance
- Researchers / Public Officials – Access arrest data for reports and transparency

Data / Sources Included:

- Arrest logs (date, time, officer, department, suspect, charges)
- Officer and arrestee personal details
- Case and charge category information
- Department and location data

INTRODUCTION

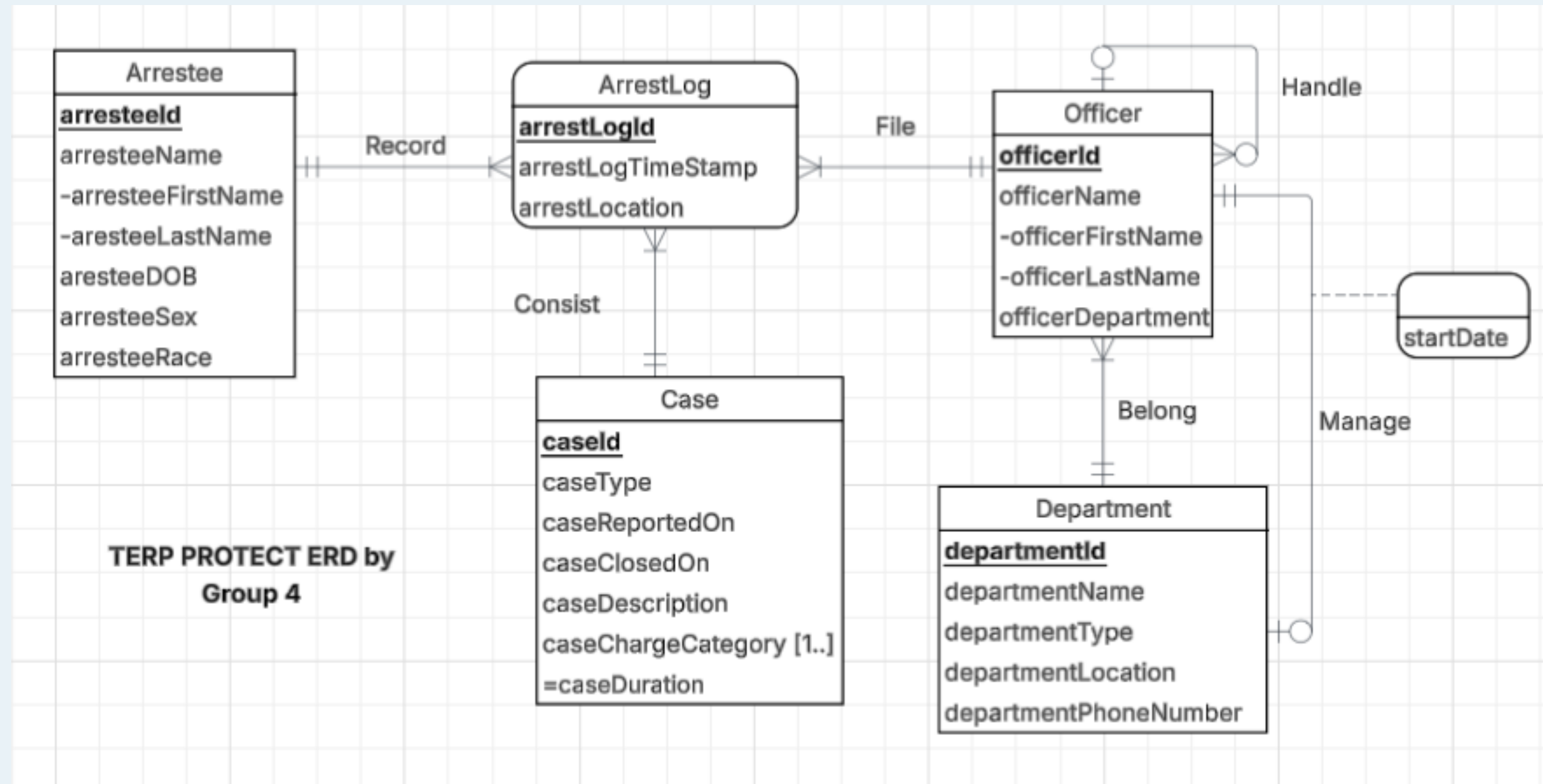
Mission Statement- To design a database system that organizes and manages Arrest logs, allowing the University of Maryland Police Department to analyze arrests by demographics, Charges, Officers, and Locations.

Mission Objectives:

- 1. Which age and sex groups have the highest number of arrests?
- 2. Determine the most common charge categories leading to arrests and how they vary over time.
- 3. Calculate the number and proportion of reported cases that did not result in any arrest
- 4. Compute and compare the average case duration across departments to identify performance differences.
- 5. Identify which departments and department heads have recorded the highest number of arrests and handled the most cases.
- 6. Determine the most frequently handled case types by each department.

ER DIAGRAM

The ER model defines how entities interact to maintain accurate and relationally consistent arrest data.



RELATIONAL

SCHEMA

- **Arrestee**(arresteeId, arresteeFirstName, arresteeLastName, arresteeDOB, arresteeSex, arresteeRace)
- **Officer**(officerId, officerFirstName, officerLastName, *departmentId*, *officerLeadId*)
- **Department**(departmentId, departmentName, departmentType, departmentLocation, departmentPhoneNumber, *officerHeadId*, startDate)
- **Case**(caseId, caseType, caseReportedOn, caseClosedOn, caseDescription)
- **caseChargeCategory**(*caseId*, caseChargeCategory)
- **ArrestLog**(*caseId*, *arresteId*, *officerId*, arrestLogId, arrestLogTimeStamp, arrestLocation)

SQL CREATE TABLES - ARRESTEE

```
CREATE TABLE [TerpProtect.Arrestee] (  
    arresteeld CHAR(9) NOT NULL,  
    arresteeFirstName VARCHAR(20) NOT NULL,  
    arresteeLastName VARCHAR(20) NOT NULL,  
    arresteeDOB DATE,  
    arresteeSex CHAR(1),  
    arresteeRace VARCHAR(20),  
    CONSTRAINT pk_Arrestee_arresteeld PRIMARY KEY (arresteeld)  
);
```

SQL CREATE TABLES - DEPARTMENT

```
CREATE TABLE [TerpProtect.Department] (  
    departmentId CHAR(9) NOT NULL,  
    departmentName VARCHAR(20) NOT NULL,  
    departmentType VARCHAR(50),  
    departmentLocation VARCHAR(100),  
    departmentPhoneNumber CHAR(10),  
    officerHeadId CHAR(9) NULL,  
    startDate DATE,  
    CONSTRAINT pk_Department_departmentId PRIMARY KEY (departmentId)
```


SQL CREATE TABLES - OFFICER

```
CREATE TABLE [TerpProtect.Officer] (  
    officerId CHAR(9) NOT NULL,  
    officerFirstName VARCHAR(20) NOT NULL,  
    officerLastName VARCHAR(20) NOT NULL,  
    departmentId CHAR(9) NOT NULL,  
    officerLeadId CHAR(9) NULL,  
    CONSTRAINT pk_Officer_officerId PRIMARY KEY (officerId),  
    CONSTRAINT fk_Officer_departmentId FOREIGN KEY (departmentId)  
        REFERENCES [TerpProtect.Department] (departmentId)  
        ON DELETE NO ACTION ON UPDATE CASCADE,  
    CONSTRAINT fk_Officer_officerLeadId FOREIGN KEY (officerLeadId)  
        REFERENCES [TerpProtect.Officer] (officerId)  
        ON DELETE NO ACTION ON UPDATE NO ACTION  
);
```


SQL CREATE TABLES - CASE

```
CREATE TABLE [TerpProtect.Case] (  
    caseId CHAR(9) NOT NULL,  
    caseType VARCHAR(30),  
    caseReportedOn DATE,  
    caseClosedOn DATE,  
    caseDescription VARCHAR(255)  
    CONSTRAINT pk_Case_caseId PRIMARY KEY (caseId)  
);
```

SQL CREATE TABLES - CASE

```
CREATE TABLE [TerpProtect.CaseChargeCategory] (  
    caseld CHAR(9) NOT NULL,  
    caseChargeCategory VARCHAR(30) NOT NULL,  
    CONSTRAINT pk_CaseChargeCategory_caseld_caseChargeCategory  
PRIMARY KEY (caseld, caseChargeCategory),  
    CONSTRAINT fk_CaseChargeCategory_caseld FOREIGN KEY (caseld)  
REFERENCES [TerpProtect.Case] (caseld)  
ON DELETE CASCADE ON UPDATE CASCADE  
);
```


SQL CREATE TABLES - ARRESTLOG

```
CREATE TABLE [TerpProtect.ArrestLog] (  
    caseld CHAR(9) NOT NULL,  
    arresteeld CHAR(9) NOT NULL,  
    officerId CHAR(9) NOT NULL,  
    arrestLogId CHAR(9) NOT NULL,  
    arrestLogTimeStamp DATETIME,  
    arrestLocation VARCHAR(50),  
    CONSTRAINT pk_ArrestLog_arrestLogId PRIMARY KEY (arrestLogId),  
    CONSTRAINT fk_ArrestLog_caseld FOREIGN KEY (caseld)  
        REFERENCES [TerpProtect.Case] (caseld)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT fk_ArrestLog_arresteeld FOREIGN KEY (arresteeld)  
        REFERENCES [TerpProtect.Arrestee] (arresteeld)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT fk_ArrestLog_officerId FOREIGN KEY (officerId)  
        REFERENCES [TerpProtect.Officer] (officerId)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

MISSION STATEMENTS

1. Which age and sex groups have the highest number of arrests?

```
SELECT
CASE
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) < 18 THEN '<18'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 18 AND 24 THEN '18-24'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 25 AND 34 THEN '25-34'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 35 AND 44 THEN '35-44'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 45 AND 54 THEN '45-54'
  ELSE '55+'
END AS ageGroup,
COALESCE(a.arresteeSex, 'Unknown') AS arresteeSex,
COUNT(*) AS arrests
FROM [TerpProtect.ArrestLog] al
JOIN [TerpProtect.Arrestee] a ON a.arresteeId = al.arresteeId
GROUP BY
CASE
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) < 18 THEN '<18'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 18 AND 24 THEN '18-24'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 25 AND 34 THEN '25-34'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 35 AND 44 THEN '35-44'
  WHEN DATEDIFF(year, a.arresteeDOB, al.arrestLogTimeStamp) BETWEEN 45 AND 54 THEN '45-54'
  ELSE '55+'
END,
COALESCE(a.arresteeSex, 'Unknown')
ORDER BY arrests DESC, ageGroup, arresteeSex;
```



The screenshot shows a SQL query results window with a table containing 7 rows of data. The table has four columns: an index, ageGroup, arresteeSex, and arrests. The data is sorted by the number of arrests in descending order.

	ageGroup	arresteeSex	arrests
1	18-24	F	30
2	25-34	M	26
3	35-44	M	18
4	25-34	F	14
5	18-24	M	6
6	<18	F	4
7	35-44	F	2



MISSION STATEMENTS

2. What are the most common charges leading to arrests, and how do they vary over time?

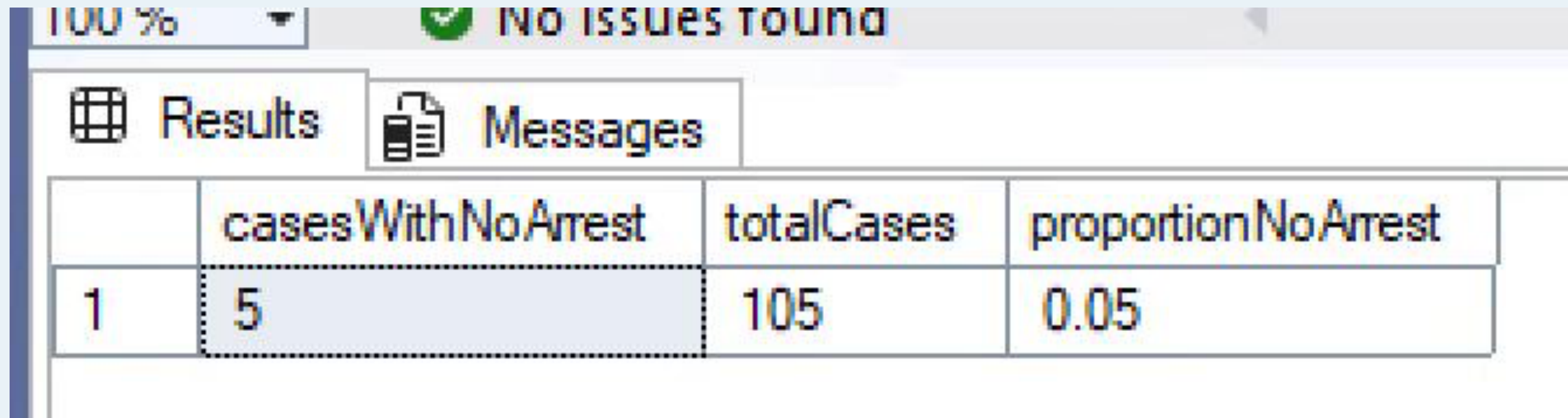
```
SELECT
    CAST(DATEFROMPARTS(YEAR(al.arrestLogTimeStamp), MONTH(al.arrestLogTimeStamp), 1) AS
DATE) AS arrestMonth,
    COALESCE(ccc.caseChargeCategory, 'Unknown') AS caseChargeCategory,
    COUNT(*) AS arrestsWithCategory
FROM [TerpProtect.ArrestLog] al
JOIN [TerpProtect.CaseChargeCategory] ccc ON ccc.caseId = al.caseId
GROUP BY
    CAST(DATEFROMPARTS(YEAR(al.arrestLogTimeStamp), MONTH(al.arrestLogTimeStamp), 1) AS
DATE),
    COALESCE(ccc.caseChargeCategory, 'Unknown')
ORDER BY arrestMonth, arrestsWithCategory DESC, caseChargeCategory;
```

	Results	Messages	
	arrestMonth	caseChargeCategory	arrestsWithCategory
51	2022-02-01	Phishing	1
52	2022-02-01	Possession	1
53	2022-02-01	Property Damage	1
54	2022-02-01	Public Disturbance	1
55	2022-02-01	Public Intoxication	1
56	2022-02-01	Reckless Driving	1
57	2022-02-01	Reckless Endangem...	1
58	2022-02-01	Resisting Detention	1
59	2022-02-01	Restraining Order Viol...	1
60	2022-02-01	Robbery	1
61	2022-02-01	Tampering with Evide...	1
62	2022-02-01	Traffic Accident	1
63	2022-02-01	Unauthorized Access	1
64	2022-02-01	Unlawful Assembly	1
65	2022-02-01	Vandalism	1
66	2022-03-01	Fraud	2
67	2022-03-01	Animal Cruelty	1
68	2022-03-01	Arson	1
69	2022-03-01	Carving Without Permit	1

MISSION STATEMENTS

3. Calculate the number and proportion of reported cases that did not result in any arrest.

```
SELECT  
    COALESCE(al.arrestLocation, 'Unknown Location') AS [Campus_Zone],  
    SELECT  
        COALESCE(SUM(CASE WHEN al.caseId IS NULL THEN 1 ELSE 0 END),  
0) AS casesWithNoArrest,  
        COALESCE(COUNT(*), 0) AS totalCases,  
        COALESCE(  
            CAST(  
                1.0 * SUM(CASE WHEN al.caseId IS NULL THEN 1 ELSE 0 END)  
                / NULLIF(COUNT(*), 0) AS DECIMAL(5,2)  
            ), 0.00  
        ) AS proportionNoArrest  
FROM [TerpProtect.Case] c  
LEFT JOIN (  
    SELECT DISTINCT caseId  
    FROM [TerpProtect.ArrestLog]  
) al ON al.caseId = c.caseId;
```



100 % NO ISSUES FOUND			
Results		Messages	
	casesWithNoArrest	totalCases	proportionNoArrest
1	5	105	0.05

MISSION STATEMENTS

4. Compute and compare the average case duration across departments to identify performance differences.

```
WITH FirstArrest AS (  
  SELECT caseId, MIN(arrestLogTimeStamp) AS firstTs  
  FROM [TerpProtect.ArrestLog]  
  GROUP BY caseId  
)  
OwnerDept AS (  
  SELECT fa.caseId, MIN(o.departmentId) AS departmentId  
  FROM FirstArrest fa  
  JOIN [TerpProtect.ArrestLog] al  
    ON al.caseId = fa.caseId AND al.arrestLogTimeStamp = fa.firstTs  
  JOIN [TerpProtect.Officer] o ON o.officerId = al.officerId  
  GROUP BY fa.caseId  
)  
LastArrest AS (  
  SELECT caseId, MAX(arrestLogTimeStamp) AS lastTs  
  FROM [TerpProtect.ArrestLog]  
  GROUP BY caseId  
)  
CaseDur AS (  
  SELECT  
    od.departmentId,  
    c.caseId,  
    CASE  
      WHEN c.caseClosedOn IS NOT NULL  
      THEN DATEDIFF(day, c.caseReportedOn, c.caseClosedOn)  
      WHEN la.lastTs IS NOT NULL  
      THEN DATEDIFF(day, c.caseReportedOn, la.lastTs)  
      ELSE NULL  
    END AS durationDays  
  FROM [TerpProtect.Case] c  
  JOIN OwnerDept od ON od.caseId = c.caseId  
  LEFT JOIN LastArrest la ON la.caseId = c.caseId  
)  
SELECT  
  COALESCE(d.departmentId, 'UNKNOWN') AS departmentId,  
  COALESCE(d.departmentName, 'Unknown Department') AS departmentName,  
  AVG(CAST(cd.durationDays AS FLOAT)) AS avgCaseDurationDays,  
  COUNT(*) AS casesCounted  
FROM CaseDur cd  
JOIN [TerpProtect.Department] d ON d.departmentId = cd.departmentId  
WHERE cd.durationDays IS NOT NULL  
GROUP BY d.departmentId, d.departmentName  
ORDER BY avgCaseDurationDays;
```

100 % NO ISSUES FOUND				
Results		Messages		
	departmentId	departmentName	avgCaseDurationDays	casesCounted
1	D108	Counter-Terror.	4.36363636363636	11
2	D103	Financial Crimes	6.27272727272727	11
3	D109	Organized Crime	7.44444444444444	9
4	D105	Forensic Science	9.16666666666667	12
5	D102	Homicide Invest.	10.9166666666667	12
6	D106	Traffic Control	11.25	12
7	D110	Comm. Outreach	11.4	10
8	D107	Special Victims	11.5454545454545	11
9	D104	Narcotics Div.	12.9166666666667	12

MISSION STATEMENTS

5. Identify which departments and department heads have recorded the highest number of arrests and handled the most cases.

```
SELECT
COALESCE(d.departmentId, 'Unknown') AS departmentId,
COALESCE(d.departmentName, 'Unknown Department') AS departmentName,
COALESCE(d.officerHeadId, 'Unassigned') AS officerHeadId,
COALESCE(head.officerFirstName + ' ' + head.officerLastName, 'No Head Assigned') AS departmentHead,
COUNT(al.arrestLogId) AS totalArrests,
COUNT(DISTINCT al.caseId) AS distinctCasesHandled
FROM [TerpProtect.Department] d
LEFT JOIN [TerpProtect.Officer] head ON head.officerId = d.officerHeadId
LEFT JOIN [TerpProtect.Officer] o ON o.departmentId = d.departmentId
LEFT JOIN [TerpProtect.ArrestLog] al ON al.officerId = o.officerId
GROUP BY
d.departmentId,
d.departmentName,
d.officerHeadId,
head.officerFirstName,
head.officerLastName
ORDER BY
totalArrests DESC,
distinctCasesHandled DESC,
d.departmentName;
```

	departmentId	departmentName	officerHeadId	departmentHead	totalArrests	distinctCasesHandled
1	D105	Forensic Science	OF0000004	Barbara Clark	12	12
2	D102	Homicide Invest.	OF0000001	Robert King	12	12
3	D104	Narcotics Div.	OF0000003	William Adams	12	12
4	D106	Traffic Control	OF0000005	Richard Lewis	12	12
5	D108	Counter-Terror.	OF0000007	Joseph Allen	11	11
6	D103	Financial Crimes	OF0000002	Linda Scott	11	11
7	D107	Special Victims	OF0000006	Susan Hall	11	11
8	D110	Comm. Outreach	OF0000009	Charles Hernandez	10	10
9	D109	Organized Crime	OF0000008	Patricia Young	9	9
10	D101	Cyber Crimes	OF0000040	Rachel Richardson	0	0

MISSION STATEMENTS

6. Most frequently handled case types by each department (top 1 per department)

```
WITH DeptCase AS (  
  SELECT  
    COALESCE(d.departmentId, 'Unknown') AS departmentId,  
    COALESCE(d.departmentName, 'Unknown Department') AS departmentName,  
    COALESCE(c.caseType, 'Unknown Case Type') AS caseType,  
    COUNT(*) AS handledCount  
  FROM [TerpProtect.ArrestLog] al  
  JOIN [TerpProtect.Officer] o ON o.officerId = al.officerId  
  JOIN [TerpProtect.Department] d ON d.departmentId = o.departmentId  
  JOIN [TerpProtect.Case] c ON c.caseId = al.caseId  
  GROUP BY d.departmentId, d.departmentName, c.caseType  
)  
,  
Ranked AS (  
  SELECT *,  
    ROW_NUMBER() OVER (  
      PARTITION BY departmentId  
      ORDER BY handledCount DESC, caseType  
    ) AS rn  
  FROM DeptCase  
)  
SELECT departmentId, departmentName, caseType, handledCount  
FROM Ranked  
WHERE rn = 1  
ORDER BY departmentName;
```

100 % NO ISSUES FOUND				
Results Messages				
	departmentId	departmentName	caseType	handledCount
1	D110	Comm. Outreach	Fraud	2
2	D108	Counter-Terror.	Assault	2
3	D103	Financial Crimes	Shoplifting	2
4	D105	Forensic Science	Cybercrime	2
5	D102	Homicide Invest.	Burglary	2
6	D104	Narcotics Div.	Animal Cruelty	2
7	D109	Organized Crime	Illegal Firearms	2
8	D107	Special Victims	Theft	2
9	D106	Traffic Control	Disorderly Conduct	2



CONCLUSION

The **Terp Protect** system efficiently centralizes and manages UMPD arrest records that were previously scattered across multiple sources. With well-defined entity relationships and business rules, it ensures data consistency, accuracy, and transparency while enabling officers and analysts to access and update information seamlessly.

Its analytical features help identify key trends such as arrest demographics, officer performance, and repeat offenses. By turning data into actionable insights, **Terp Protect** enhances decision-making, accountability, and overall campus safety at the University of Maryland.



THANK YOU